

Marketplace Multivendeur + Live Shopping

Version peaufinée — Cahier des charges pour génération de code par IA

20 août 2026

- [0. Notes de révision \(à lire avant utilisation\)](#)
- [1. Feuille de route et périmètre \(MVP vs V2\)](#)
- [2. Objectif général](#)
- [3. Génération, propriété et hébergement](#)
- [4. Stack technique \(versions cibles à fixer explicitement\)](#)
- [5. Architecture](#)
- [6. Utilisateurs et authentification](#)
- [7. Politique de confidentialité, CGU et documents juridiques](#)
- [8. Vendeurs et boutiques](#)
- [9. Catalogue multivendeur](#)
- [10. Commandes multivendeurs](#)
- [11. Paiement — architecture préparée \(non implémenté\)](#)
- [12. Commissions](#)
- [13. Livraison](#)
- [14. Mini réseau social produit](#)
- [15. Notifications](#)
- [16. Live Shopping](#)
- [17. Dashboards](#)
- [18. Sécurité générale](#)
- [19. API REST \(indicatif, sous /api/v1 \)](#)
- [20. Base de données — entités principales](#)
- [21. Design, SEO, performance, accessibilité](#)
- [22. Hébergement à coût 0 \(point de départ\)](#)
- [23. Docker et variables d'environnement](#)
- [24. Tests — et critères d'acceptation](#)
- [25. README et livrables](#)
- [26. Contrôle qualité avant livraison](#)
- [27. Objectif final](#)
- [28. Résumé des changements apportés à ce prompt](#)

0. Notes de révision (à lire avant utilisation)

Ce document est une version peaufinée du prompt original. Le contenu métier a été conservé presque intégralement ; les changements portent sur la structure, la levée d'ambiguïtés, et l'ajout d'éléments manquants mais nécessaires à une génération de code exploitable. Un résumé des changements se trouve en fin de document (section 21).

Point le plus important : ce cahier des charges décrit un projet de plusieurs dizaines de milliers de lignes de code (marketplace + temps réel + Live Shopping + juridique). Aucune IA ne peut le générer intégralement, testé et cohérent, en une seule réponse. Deux options :

- Recommandé — génération par lots.** Utiliser ce document comme référence stable, puis le soumettre par modules successifs dans des prompts séparés (voir la feuille de route en section 1), en demandant à chaque étape de respecter l'arborescence et les conventions définies ici.
- Génération en un bloc.** Accepter que le premier résultat sera un squelette architectural (structure de dossiers, entités, endpoints principaux, un flux complet de bout en bout) plutôt qu'une implémentation exhaustive de chaque fonctionnalité listée.

1. Feuille de route et périmètre (MVP vs V2)

1.1 Phase 0 — Socle

Auth (JWT + refresh), rôles, utilisateurs, boutiques, catégories, produits, panier, commandes multivendeur, Docker Compose, README, `.env.example`.

1.2 Phase 1 — Marketplace opérationnelle

Commissions, livraison (statuts + zones), carte interactive, avis, favoris, mini réseau social produit, notifications, stock, promotions, dashboards vendeur/admin, documents juridiques + consentements (voir 1.4).

1.3 Phase 2 — Live Shopping

Module `live` complet (sessions, produits épinglés, chat, WebSocket, modération, analytics).

1.4 Transversal dès la Phase 0

Sécurité (RBAC, isolation des données par vendeur), audit log, pages légales (mêmes vides/brouillon au départ), variables d'environnement, tests.

Non-objectifs explicites (hors périmètre) : - Paiement en ligne réel (préparé uniquement, voir section 9). - Diffusion vidéo à grande échelle via le backend applicatif (abstraction uniquement, voir section 13.9). - Conformité juridique certifiée automatiquement (voir section 4). - Traduction multilingue complète de l'interface (prévoir la structure `i18n`, pas le contenu).

2. Objectif général

Développer une plateforme web e-commerce multivendeur moderne, professionnelle, responsive, sécurisée, performante, maintenable et évolutive, mettant en relation des **acheteurs**, des **vendeurs**, et un **administrateur** qui supervise l'ensemble de la marketplace.

Les visiteurs consultent librement le catalogue sans compte. Toute action nécessitant une identité ou modifiant des données requiert une authentification.

Le projet est une véritable marketplace, pas un site vitrine.

3. Génération, propriété et hébergement

L'IA génère uniquement : code source, fichiers de configuration, scripts, documentation, migrations, fichiers Docker, tests, et tout fichier nécessaire au fonctionnement du projet.

L'IA ne déploie jamais automatiquement l'application. Le porteur du projet effectue lui-même : création du dépôt GitHub, configuration des services, PostgreSQL, stockage des images, variables d'environnement, déploiement frontend/backend, configuration du domaine, maintenance.

Le projet doit être totalement indépendant de la plateforme d'IA ayant généré le code : aucune dépendance à un service propriétaire de génération, aucune clé ou SDK spécifique à l'outil utilisé.

4. Stack technique (versions cibles à fixer explicitement)

Frontend - React 18+, Vite, TypeScript (strict mode) - Tailwind CSS + DaisyUI (ou équivalent) - React Router, TanStack Query, Axios - Bibliothèque de graphiques (ex. Recharts) - Leaflet + OpenStreetMap (ou MapLibre) — pas de dépendance obligatoire à Google Maps

Backend - Java 21 (LTS), Spring Boot 3.x - Spring Security + JWT, REST API - JPA/Hibernate, Bean Validation - Migrations via Flyway ou Liquibase (choisir un seul outil et s'y tenir)

Base de données - PostgreSQL 16+

Stockage images - Cloudinary ou stockage compatible S3 (abstraction `StorageProvider` pour permettre de changer de fournisseur)

API - Swagger / OpenAPI, versionnement des routes sous `/api/v1`

DevOps - Docker, Docker Compose, Git, GitHub - GitHub Actions pour lint + tests automatiques sur chaque pull request (sans déploiement automatique)

Internationalisation - Langue par défaut : français. Prévoir une structure de clés de traduction (frontend) et de messages (backend) permettant d'ajouter d'autres langues plus tard, sans traduire réellement tout le contenu dès maintenant.

5. Architecture

Frontend : `components`, `pages`, `layouts`, `hooks`, `services`, `contexts`, `types`, `utils`, `routes`, `guards`, `assets`, `features`,

realtime.

Backend : controller, service, repository, entity, dto, mapper, security, configuration, exception, validation, notification, payment, commission, delivery, live, moderation.

Respecter : SOLID, DRY, Clean Code, séparation des responsabilités, architecture modulaire, faible couplage, testabilité, maintenabilité, évolutivité.

Conventions API : pagination par défaut (page, size, max size autorisé), tri via sort, réponses d'erreur structurées et centralisées (un seul format d'erreur JSON pour toute l'API), codes HTTP cohérents.

6. Utilisateurs et authentification

Rôles : BUYER, SELLER, ADMIN, éventuellement MODERATOR.

Accès visiteur (sans compte) : accueil, catégories, recherche, fiches produits, boutiques, discussions publiques, avis, Lives publics, catalogue.

Compte obligatoire pour : commander, ajouter aux favoris, commenter, réagir, publier un avis, participer au chat d'un Live, enregistrer une localisation, gérer son profil, toute action privée ou modifiant des données.

À prévoir : inscription, connexion, déconnexion, JWT + refresh token sécurisé (rotation + invalidation), récupération et changement de mot de passe, vérification email, gestion des rôles, protection des routes (frontend et backend), expiration des tokens, gestion des sessions, rate limiting sur les endpoints sensibles (login, inscription, réinitialisation de mot de passe).

7. Politique de confidentialité, CGU et documents juridiques

Pages publiques : /privacy-policy, /terms, /sales-terms, /cookie-policy, /refund-policy, /seller-terms, /legal.

Ces pages sont reliées au footer, à l'inscription, aux paramètres du compte, aux commandes, et au Dashboard Admin.

Avertissement à conserver tel quel dans le projet livré : l'IA ne certifie aucune conformité juridique automatique. Les textes définitifs doivent être adaptés au pays d'exploitation, aux données réellement traitées, et validés juridiquement par un professionnel si nécessaire.

Entité LegalDocument : id, type, title, version, content, status, publishedAt, effectiveAt, createdAt, updatedAt, createdBy, updatedBy, changeSummary.

Types : PRIVACY_POLICY, TERMS_OF_USE, SALES_TERMS, COOKIE_POLICY, REFUND_POLICY, SELLER_TERMS. Statuts : DRAFT, PUBLISHED, ARCHIVED. Toutes les versions publiées sont conservées.

Entité LegalConsent : id, userId, legalDocumentId, documentVersion, acceptedAt, ipAddress (si légalement justifié), userAgent (si légalement justifié).

À l'inscription : consentement explicite aux conditions et à la politique de confidentialité, jamais précoché. Lorsqu'une nouvelle version obligatoire est publiée, prévoir un mécanisme configurable de nouvelle acceptation.

/admin/legal permet de : créer, modifier un brouillon, prévisualiser, publier, archiver, versionner, consulter l'historique, renseigner les changements. Toute action sensible est tracée dans un **audit log**.

La politique de confidentialité doit pouvoir documenter, selon les données réellement utilisées : comptes, commandes, localisation, données techniques, images, commentaires, avis, Live Shopping, notifications, stockage, services tiers, paiement futur, droits des utilisateurs, conservation, sécurité, cookies, contact.

/account/privacy permet : consultation des données, modification, export lorsque prévu, suppression, préférences, historique des consentements. La suppression/anonymisation respecte les données devant être légalement conservées.

8. Vendeurs et boutiques

Chaque vendeur possède un compte, un profil public et une boutique avec : nom, logo, bannière, description, slogan, coordonnées, réseaux sociaux, horaires, localisation, statut, produits, commandes, statistiques, commissions, Lives.

Personnalisation sauvegardée en base et appliquée au profil public : logo, nom, slogan, description, couleurs, bannière, image de profil, réseaux sociaux, téléphone, email, adresse, horaires.

9. Catalogue multivendeur

Chaque produit appartient obligatoirement à un vendeur. Structure : Vendeur → Boutique → Produits → Catégories.

Produit : nom, description, prix (devise par défaut FCFA/XOF, stocké en unité mineure entière pour éviter les erreurs d'arrondi ; prévoir un champ devise pour une évolution future), ancien prix, promotion, images, catégorie, sous-catégorie, marque, variantes, couleurs, tailles, stock, référence, statut, dates de création/mise à jour.

À prévoir : recherche (full-text PostgreSQL en V1 ; abstraction permettant de brancher un moteur dédié comme Meilisearch/Elasticsearch en V2), filtres, tri, pagination, nouveautés, promotions, populaires, similaires, produits d'un vendeur, partage.

9.1 Catégories

Catégorie, sous-catégorie, image, description, statut. Admin : créer, modifier, supprimer, réorganiser, activer/désactiver.

10. Commandes multivendeurs

Le panier peut contenir plusieurs vendeurs.

- `Order` = commande globale
- `SellerOrder` = sous-commande par vendeur
- `OrderItem` = lignes de commande

La commande globale expose : total global, sous-total par vendeur, frais de livraison, statut global, historique. Chaque vendeur ne voit que ses `SellerOrder` ; l'administrateur voit tout.

Statuts : `PENDING`, `CONFIRMED`, `PREPARING`, `READY_FOR_DELIVERY`, `DELIVERING`, `DELIVERED`, `CANCELLED`. L'historique de chaque changement de statut est conservé.

11. Paiement — architecture préparée (non implémenté)

Le paiement en ligne n'est **pas** implémenté. Actuellement : commande confirmée → paiement physique → livraison physique.

Créer une abstraction `PaymentService` / `PaymentProvider`.

Statuts : `PAYMENT_PENDING`, `PAYMENT_SUCCESS`, `PAYMENT_FAILED`, `PAYMENT_REFUNDED`, `PAYMENT_CANCELLED`.

Préparer des adaptateurs futurs pour : TMoney, Flooz, cartes bancaires, PayPal, autres fournisseurs. **Ne jamais simuler une transaction réelle.**

Commentaires obligatoires dans le code aux points d'intégration : `TODO PAYMENT`, `TODO PAYMENT WEBHOOK`, `TODO COMMISSION SETTLEMENT`.

Les commissions ne doivent pas être couplées à un fournisseur de paiement particulier.

12. Commissions

Taux configurable par vendeur (taux global, éventuellement taux par catégorie).

À calculer et stocker : montant brut, commission plateforme, montant net vendeur, statut, période, référence commande.

Exemple : 100 000 FCFA, 10 % → 10 000 FCFA de commission, 90 000 FCFA pour le vendeur.

Admin : total commissions, commissions par vendeur, revenus plateforme, montants dus, historique, filtres. **Vendeur** : chiffre d'affaires, commissions, montant net, historique, commissions par commande.

13. Livraison

Actuellement physique. Créer `DeliveryService`, `DeliveryStatus`, `DeliveryZone`, `DeliveryFee`.

Vendeur : zones, frais, délai, disponibilité. **Livraison** : adresse, latitude, longitude, instructions, vendeur, acheteur, statut.

Statuts : `DELIVERY_PENDING`, `PREPARING`, `IN_TRANSIT`, `DELIVERED`, `CANCELLED`. Commentaire `TODO DELIVERY` pour une future intégration logistique.

13.1 Carte interactive

Couverture minimale : région Maritime du Togo. Leaflet/OpenStreetMap ou MapLibre privilégiés ; Google Maps non obligatoire.

Fonctionnalités : carte, zoom, déplacement, recherche, marqueur, géolocalisation avec permission explicite, sélection manuelle, déplacement du marqueur, latitude/longitude, adresse, sauvegarde.

Acheteur : nom, téléphone, adresse, ville, GPS, instructions. **Vendeur** : localisation boutique, adresse, GPS, zone de livraison.

Préparer : calcul de distance, frais selon distance, zones, estimation de délai.

14. Mini réseau social produit

Sous chaque produit : réactions (LIKE , LOVE , HAHA , WOW , SAD , ANGRY), likes, commentaires (réponses imbriquées, pagination, suppression par l’auteur, modération vendeur/admin, signalement), partage.

14.1 Avis

Acheteur authentifié : note 1-5, commentaire, image facultative, date, modération. Afficher moyenne, nombre, répartition.

14.2 Favoris

Ajouter, retirer, liste des favoris.

14.3 Panier

Support multivendeur. Afficher par vendeur : produits, quantités, sous-total, livraison, puis un total général.

14.4 Promotions

Réduction produit, catégorie, coupon, vente flash, dates, limites d’utilisation.

14.5 Stock

Stock, seuil faible, rupture, historique, entrées, sorties, ajustement. Le vendeur est notifié automatiquement. Les mises à jour de stock doivent être transactionnelles pour éviter les ventes en survente (voir aussi section 16, gestion du stock concurrent pendant un Live).

15. Notifications

Entité `Notification` : type, titre, message, utilisateur, date, lu/non lu, référence.

Événements notifiant le vendeur : nouvelle commande, confirmation, annulation, livraison, modification, rupture, stock faible, nouveau commentaire, nouvel avis, nouvelle réaction, nouveau Live ou événement important.

Canaux à préparer (abstraction, pas nécessairement tous actifs en V1) : notifications internes, email, push, SMS.

16. Live Shopping

Objectif : permettre à un vendeur de présenter ses produits en temps réel, interagir avec les spectateurs et vendre directement pendant la diffusion — inspiré des mécaniques UX connues (vidéo verticale, interaction temps réel, produits épinglés, chat, réactions, achats rapides, modération, statistiques), **sans copier l’interface, le code, la marque ou tout élément propriétaire d’une plateforme existante.**

16.1 Architecture

Module indépendant : `frontend/src/features/live/` et `backend/.../live/`.

Entités : `LiveSession`, `LiveProduct`, `LiveViewer`, `LiveMessage`, `LiveReaction`, `LiveOrderEvent`, `LiveModerationAction`, `LiveAnalytics`.

`LiveSession` : `id`, `sellerId`, `title`, `description`, `thumbnail`, `status`, `startedAt`, `endedAt`, `scheduledAt`, `viewerCount`, `peakViewerCount`, `productCount`, `chatEnabled`, `reactionsEnabled`, `createdAt`.

Statuts : `SCHEDULED`, `LIVE`, `PAUSED`, `ENDED`, `CANCELLED`.

16.2 Fonctionnalités vendeur pendant un Live

Démarrer, arrêter, mettre en pause ; afficher/épingler/retirer un produit ; réordonner les produits ; afficher prix et promotion ; consulter le stock ; voir les commandes liées au Live ; gérer les messages (voir, supprimer) ; bloquer un utilisateur ; ralentir le chat ; activer/désactiver les réactions ; mettre en avant une annonce ; voir spectateurs, produits les plus consultés, ajouts au

panier, ventes générées.

Le vendeur ne peut sélectionner que ses propres produits. Stock et disponibilité affichés proviennent des données réelles de la marketplace (pas de duplication d'état).

16.3 Gestion temps réel des produits

Panneau « Produits du Live » : recherche, ajout, retrait, réordonnancement, épinglage, consultation du stock actuel.

Produit épinglé → visible de tous les spectateurs avec prix, promotion, stock, bouton d'action. Si le stock atteint zéro, l'interface se met à jour en temps réel. Les contrôles de cohérence de stock se font via des transactions côté backend (le frontend ne décrémente jamais le stock de son propre chef).

16.4 Achat pendant le Live

Le spectateur authentifié : ouvre la fiche produit, ajoute au panier, achète, consulte les détails, revient immédiatement au Live. Le Live ne contourne jamais le circuit normal de commande. Les commandes issues d'un Live conservent une référence `liveSessionId`.

Statistiques à prévoir : vues produit, clics, ajouts panier, commandes, chiffre d'affaires, conversion.

16.5 Chat temps réel

Messages texte, réactions, réponses (optionnel), compteur, modération, blocage, signalement, limitation anti-spam. Ne pas conserver indéfiniment des données inutiles (politique de rétention à définir, cohérente avec la politique de confidentialité — section 7).

16.6 WebSocket / temps réel

Architecture WebSocket (STOMP si pertinent avec Spring), avec Redis en option pour une architecture distribuée future. Événements typés, par exemple : `LIVE_STARTED`, `LIVE_ENDED`, `VIEWER_JOINED`, `VIEWER_LEFT`, `PRODUCT_PINNED`, `PRODUCT_UNPINNED`, `STOCK_UPDATED`, `PRICE_UPDATED`, `NEW_MESSAGE`, `MESSAGE_DELETED`, `REACTION_ADDED`, `ORDER_CREATED`, `ORDER_STATUS_CHANGED`.

Le backend reste la source de vérité. Le frontend ne considère jamais une donnée reçue par WebSocket comme valide sans validation serveur correspondante.

16.7 Interface spectateur

Mobile-first. Vidéo principale, informations vendeur, titre, nombre de spectateurs, chat, réactions, produit épinglé, bouton acheter, liste de produits, partage, signalement. En desktop : vidéo, chat, produits, informations vendeur, disposés en colonnes.

16.8 Programmation des Lives

Le vendeur programme : date, heure, titre, description, miniature, produits prévus. Les visiteurs voient les Lives à venir, peuvent activer un rappel, et accèdent à la boutique du vendeur. Notification avant le Live.

16.9 Analytics Live

Vendeur : spectateurs uniques, pic, durée, durée moyenne, messages, réactions, clics produits, ajouts panier, commandes, CA, conversion, produit le plus performant. **Admin** : nombre de Lives, Lives actifs, vendeurs actifs, audience, ventes Live, signalements, performances globales.

16.10 Médias Live

Abstraction `LiveStreamingProvider` — ne pas enfermer l'application dans un fournisseur unique. Préparer : source vidéo, URL de stream, statut, miniature, métadonnées. Commentaires `TODO` pour l'intégration future d'un vrai fournisseur de streaming/CDN. **Un simple serveur Spring Boot ne doit pas être supposé capable de transporter seul toute la vidéo à grande échelle ;** documenter cette limite dans le README (voir section 20).

16.11 Modération Live

Admin et vendeur : supprimer message, bloquer utilisateur, ralentir/fermer le chat, signaler, interrompre un Live selon permissions. Prévoir : mots interdits configurables, anti-spam, rate limiting, journal de modération.

16.12 Sécurité Live

Vérifier systématiquement : propriétaire du Live, propriétaire des produits, permissions, accès aux commandes et statistiques, modération, endpoints WebSocket. **Un vendeur ne peut jamais contrôler le Live d'un autre vendeur.**

17. Dashboards

17.1 Dashboard vendeur avancé

Chiffre d'affaires, commandes (par statut), produits, rupture, stock faible, commissions, net, avis, commentaires, notifications, performances Live.

Graphiques : ventes jour/semaine/mois/année, produits populaires, revenus, commissions, panier moyen, commandes, conversion (si données disponibles), performances des Lives.

Gestion : produits, commandes, clients, commentaires, avis, stock, promotions, boutique, paramètres, notifications, Lives.

17.2 Dashboard administrateur

Vendeurs, acheteurs, produits, commandes, chiffre d'affaires, commissions, revenus plateforme, annulations, ruptures, statistiques, Lives actifs, statistiques Live, signalements.

Gestion : vendeurs, acheteurs, produits, catégories, commandes, commissions, commentaires, avis, signalements, promotions, notifications, documents juridiques, paramètres.

Pouvoirs admin : activer/désactiver vendeur, suspendre compte, modifier commission, modérer produit/commentaire/avis, gérer catégories, interrompre un Live si nécessaire.

18. Sécurité générale

Spring Security, JWT, RBAC, validation des entrées, CORS, protection contre les injections, gestion centralisée des exceptions, contrôle systématique de propriété des ressources, rate limiting, audit logs pour les opérations sensibles.

Règle fondamentale, non négociable : un vendeur A ne voit ni ne modifie jamais les données privées d'un vendeur B ; un acheteur n'accède qu'à ses propres données ; l'admin possède les permissions globales. Cette isolation doit être testée explicitement (voir section 20).

19. API REST (indicatif, sous `/api/v1`)

`/auth` , `/users` , `/buyers` , `/sellers` , `/shops` , `/products` , `/categories` , `/cart` , `/orders` , `/seller-orders` , `/payments` , `/commissions` , `/deliveries` , `/notifications` , `/comments` , `/reactions` , `/reviews` , `/favorites` , `/locations` , `/dashboard` , `/settings` , `/legal` , `/live` , `/live-sessions` , `/live-products` , `/live-messages` , `/live-reactions` , `/live-analytics` .

Documentation via Swagger/OpenAPI, accessible en développement uniquement (désactivée ou protégée en production).

20. Base de données — entités principales

User , Role , BuyerProfile , SellerProfile , Shop , Product , ProductImage , Category , SubCategory , ProductVariant , Cart , CartItem , Order , SellerOrder , OrderItem , Payment , Commission , Delivery , Address , Location , Notification , Comment , Reaction , Review , Favorite , Promotion , Coupon , StockMovement , LegalDocument , LegalConsent , AuditLog , LiveSession , LiveProduct , LiveViewer , LiveMessage , LiveReaction , LiveOrderEvent , LiveModerationAction , LiveAnalytics .

Ajouter relations, contraintes, index, timestamps, UUID lorsque pertinent, unicité, clés étrangères. **Éviter les suppressions en cascade dangereuses sur les données commerciales et juridiques** (préférer un archivage/soft delete pour Order , LegalDocument , LegalConsent , Commission).

21. Design, SEO, performance, accessibilité

Design : moderne, premium, responsive, mobile-first, animations fluides, mode clair/sombre, identité cohérente. Compatible smartphone, tablette, ordinateur.

Accessibilité : viser WCAG 2.1 niveau AA (contrastes, navigation clavier, attributs ARIA sur les composants interactifs, notamment le chat Live et les formulaires).

SEO et performance : URLs propres, métadonnées, titres dynamiques, descriptions, Open Graph, lazy loading, optimisation images, pagination, cache, chargement progressif, compression, code splitting.

Observabilité minimale : endpoint de santé (`/actuator/health`), logs structurés, identifiants de corrélation pour le suivi des requêtes.

22. Hébergement à coût 0 (point de départ)

Architecture pouvant démarrer sur des offres gratuites (sous réserve de leurs limites au moment du déploiement), par exemple : GitHub (code), Vercel (frontend), Render/Koyeb (backend), Supabase/Neon (PostgreSQL), Cloudinary (images).

Pour le Live Shopping, le streaming vidéo temps réel nécessitera probablement un service spécialisé à terme — ne pas supposer qu'un hébergement gratuit classique suffira à grande échelle. Le projet doit pouvoir migrer vers une infrastructure payante sans réécriture majeure.

23. Docker et variables d'environnement

Fournir : Dockerfile frontend, Dockerfile backend, `docker-compose.yml`, PostgreSQL local, configuration réseau, variables d'environnement. Démarrage local complet via Docker Compose.

Aucun secret dans le code. Variables notamment : `DATABASE_URL`, `DATABASE_USERNAME`, `DATABASE_PASSWORD`, `JWT_SECRET`, `CLOUDINARY_URL` (ou `STORAGE_PROVIDER` + variables S3 équivalentes), `CORS_ALLOWED_ORIGINS`, `APP_BASE_URL`, `FRONTEND_URL`, `PAYMENT_PROVIDER`, `PAYMENT_API_KEY`, `MAP_PROVIDER`, `LEGAL_COMPANY_NAME`, `LEGAL_CONTACT_EMAIL`, `LEGAL_CONTACT_ADDRESS`, `LEGAL_COUNTRY`, `PRIVACY_POLICY_VERSION`, `TERMS_VERSION`.

Créer uniquement `.env.example`, sans secret réel.

24. Tests — et critères d'acceptation

Créer des tests unitaires (services, repositories importants, contrôleurs, sécurité, permissions) et d'intégration.

Critères d'acceptation à valider explicitement par des tests, module par module : - isolation des données entre vendeurs (un vendeur A ne peut jamais lire/modifier les ressources du vendeur B) ; - permissions par rôle sur chaque endpoint sensible ; - calcul de commission exact (montant brut, commission, net) ; - panier et commande multivendeur (répartition correcte par `SellerOrder`) ; - stock concurrent (pas de survente lors d'achats simultanés, y compris pendant un Live) ; - annulation de commande et effets en cascade (stock, notification) ; - notifications déclenchées aux bons événements ; - documents juridiques : versionnement, consentement obligatoire, non pré-cochage ; - Live Shopping : ajout/retrait de produit en direct, cohérence du stock affiché, chat, modération, commande avec `liveSessionId` correct ; - WebSocket : le serveur rejette toute donnée non validée provenant d'un client.

25. README et livrables

README professionnel couvrant : présentation, fonctionnalités, architecture, stack, prérequis, installation locale, configuration `.env`, PostgreSQL, Cloudinary/S3, carte, WebSocket, Live Streaming (et ses limites, voir 16.10), paiement futur, livraison, commissions, tests, Docker, déploiement manuel, hébergement à coût 0, migration vers infrastructure payante, sécurité, sauvegardes, documents juridiques, checklist de production.

Livrables : arborescence complète, code source frontend et backend, migrations (Flyway ou Liquibase), Dockerfiles, `docker-compose.yml`, `.env.example`, Swagger, README, scripts d'installation/démarrage/sauvegarde, tests, données de démonstration optionnelles, documentation d'architecture et d'API.

Aucune maquette ni pseudo-code : chaque fonctionnalité demandée doit être réellement reliée au backend, à la base de données et à l'interface.

26. Contrôle qualité avant livraison

Avant de considérer un lot terminé, vérifier : cohérence des entités, relations JPA, DTO, migrations, endpoints, permissions, variables d'environnement, appels frontend/backend, WebSockets, transactions, calculs de commission, isolation multivendeur, gestion du stock concurrent, tests, Docker, README.

Ne pas inventer de fonctionnalités non demandées qui complexifieraient inutilement le projet. Pour toute intégration externe nécessaire mais non activée : créer une abstraction claire, un adaptateur vide/mock réservé aux tests, et un commentaire `TODO` précisant où l'intégration réelle devra être ajoutée.

27. Objectif final

Produire une base de code professionnelle, complète, indépendante de la plateforme d'IA, prête à être exécutée localement avec

Docker puis déployée manuellement — une marketplace multivendeur avec acheteurs, vendeurs, administrateur, boutiques personnalisables, catalogue, panier multivendeur, commandes, commissions, livraison, carte de localisation, notifications, réseau social produit, avis, dashboards vendeur et admin avancés, paiements préparés mais non activés, documents juridiques versionnés et consentements, Live Shopping temps réel avec gestion des produits et commandes pendant les Lives, architecture temps réel évolutive, hébergement initial à coût 0 lorsque possible, et une infrastructure que le porteur du projet peut déployer et maintenir lui-même.

28. Résumé des changements apportés à ce prompt

1. Ajout d'une feuille de route en phases (section 1) pour rendre la génération réaliste et éviter un résultat superficiel sur l'ensemble du périmètre.
2. Ajout d'une section "non-objectifs" explicite.
3. Versions cibles précisées pour la stack (Java 21, Spring Boot 3.x, PostgreSQL 16, React 18) pour éviter les incohérences de génération.
4. Choix explicite à faire entre Flyway et Liquibase (le prompt original citait les deux sans trancher).
5. Ajout des conventions API (/api/v1 , pagination, format d'erreur unique).
6. Ajout d'une stratégie de recherche progressive (full-text PostgreSQL puis moteur dédié) plutôt qu'une simple mention de "recherche".
7. Ajout d'une remarque sur le stockage monétaire en unité mineure entière (éviter les erreurs d'arrondi en FCFA).
8. Ajout d'une section accessibilité (WCAG 2.1 AA) et d'une section observabilité minimale (health check, logs structurés).
9. Ajout de CI (GitHub Actions pour lint/tests) sans introduire de déploiement automatique, ce qui reste conforme à l'exigence d'indépendance vis-à-vis de la plateforme d'IA.
10. Reformulation de la section tests en critères d'acceptation vérifiables, module par module.
11. Précision sur le soft delete / archivage pour les données commerciales et juridiques plutôt qu'une suppression en cascade.
12. Clarification de la protection de Swagger en production.
13. Ajout d'une structure i18n minimale sans exiger une traduction complète immédiate.
14. Suppression des redondances entre sections (ex. panier, avis, favoris regroupés sous une même section 14).